

OS_2022-01-25

1) Synchronisation mit Semaphoren (30)

Ein Softwarepaket aus vier Prozessen soll eine Anlage simulieren, die Kartons mit Würfeln und Murmeln befüllt. Zwei gleichartige Prozesse simulieren den An- und Abtransport von Kartons zur bzw. von der Befüllanlage. Je ein Prozess kontrolliert das Einfüllen von Murmeln bzw. Würfeln in einen bereitstehenden Karton.

Ergänzen Sie die gegebenen Codestücke mit *Semaphoroperationen*, um die Prozesse entsprechend der angeführten Bedingungen zu synchronisieren. Das Verwenden von globalen Variablen zur Synchronisation oder Kommunikation zwischen Prozessen ist nicht erlaubt.

- Die beiden Prozesse für den An- und Abtransport der Kartons sollen Roboter simulieren. Jeder Roboter hat einen eigenen Vorrat an leeren Kartons und einen eigenen Ablageplatz für volle Kartons und führt wiederholt folgende Aktionen durch:
 - Karton vom Vorrat holen: *karton_aufnehmen()*.
 - Karton zur Befüllanlage transportieren und öffnen: *karton_bereitstellen()*.
 - Karton verschließen und von der Befüllanlage nehmen: *karton_entfernen()*.
 - Karton zum Ablageplatz abtransportieren: *karton_abtransportieren()*.
- Der Prozess Murmelspender ruft wiederholt die Funktion *Murmel_liefern()* auf. Jeder Aufruf der Funktion lässt eine Murmel in den Karton auf der Befüllanlage fallen.
- Der Prozess Würfelspender ruft wiederholt die Funktion *Wuerfel_liefern()* auf. Jeder Aufruf der Funktion lässt einen Würfel in den Karton auf der Befüllanlage fallen.
- Die Prozesse sind so zu synchronisieren, dass die Parallelität ihrer Aktionen die folgenden Regeln erfüllt, darüberhinaus aber möglichst wenig eingeschränkt wird.
- Auf dem Befüllplatz der Anlage ist Platz für einen Karton. Die Reihenfolge, in der die Roboter die Befüllanlage mit Kartons versorgen, ist nicht einzuschränken.
- Jeder Karton ist mit N Murmeln und $N + K$ Würfeln zu befüllen. Dabei sollen zuerst N Murmeln bzw. Würfel abwechselnd, beginnend mit einem Würfel, und dann die restlichen K Würfel in die Kartons gefüllt werden.

(a) Initialisierungen (vor Start der Prozesse)

```

init(würfel, 0);
init(murmel, 0)           init(platzfrei, 1)
    init(würfelcount, 0);
    init(würfel_mutex, 1);
    init(murmel_mutex, 0);

```

In der Folge sind Codestücke für die zu synchronisierenden Prozesse gegeben, die bereits alle notwendigen Funktionsaufrufe enthalten. Ergänzen Sie in diesen Codestücken die fehlenden Semaphoroperationen zur Synchronisation.

(b) Prozesse für die Befüllung

```
/** Murmelspender **/
```

```
do {
```

```

    wait(murmel)
    wait(murmel_mutex)

```

```
    Murmel_liefern();
```

```
    signal(würfel_mutex)
```

```
} while(1);
```

```
/** Wuerfelspender **/
```

```
do {
```

```

    wait(würfel)
    wait(würfel_mutex)

```

```
    Wuerfel_liefern();
```

```

    signal(murmel_mutex)
    signal(würfelcount)

```

```

    wait(würfel)
    wait(würfel_mutex)

```

```
    Wuerfel_liefern();
```

```

    signal(murmel_mutex)
    signal(würfelcount)

```

```
} while(1);
```

?

(c) Prozesse für Roboter

```
/** Roboter R1 **/
```

```
do {
```

```
    karton_aufnehmen(R1);
```

```
/** Roboter R2 **/
```

```
do {
```

```
    karton_aufnehmen(R2);
```

```
wait(platzfrei)
```

```

karton_bereitstellen(R1);
for (int i = 0; i < N+K; i++) {
    signal(würfel)
    if (i < N) signal(murmel);
    if (i >= N) signal(würfel_mutex);
}

```

```
for (int i = 0; i < N + K; i++;) wait(würfelcount);
```

```
karton_entfernen(R1);
```

```
signal(platzfrei)
```

```
karton_abtransportieren(R1);
```

```
} while (1)
```

```
wait(platzfrei)
```

```

karton_bereitstellen(R2);
for (int i = 0; i < N+K; i++) {
    signal(würfel)
    if (i < N) signal(murmel);
    if (i >= N) signal(würfel_mutex);
}

```

```
for (int i = 0; i < N + K; i++;) wait(würfelcount);
```

```
karton_entfernen(R2);
```

```
signal(platzfrei)
```

```
karton_abtransportieren(R2);
```

```
} while(1);
```


2) Scheduling (30)

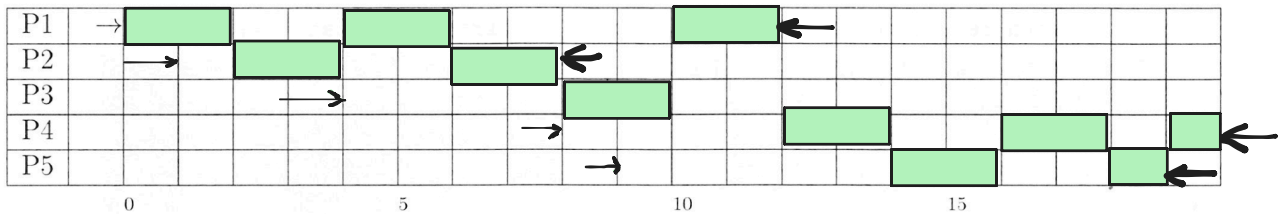
Schedulen Sie das nebenstehende Taskset mit den Verfahren *Round Robin* (RR), *Shortest Remaining Time* (SRT), bzw. *Highest Response Ratio Next* (HRRN). Der Overhead für den Taskwechsel ist vernachlässigbar. Bei RR beträgt die Zeitscheibenlänge 2 Zeiteinheiten.

Task	Arrival Time	Service Time
P1	0	6
P2	1	4
P3	4	2
P4	8	5
P5	9	3

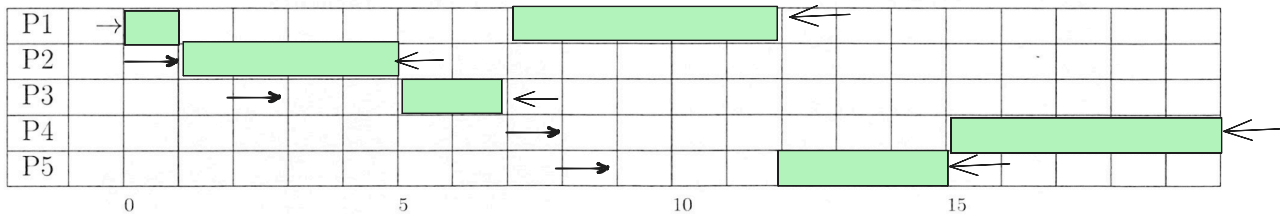
Fallen beim RR-Scheduling das Ende einer Zeitscheibe und die Ankunft eines Tasks zeitlich zusammen, so behandeln Sie den unterbrochenen Task vor dem neuen. Bei SRT bzw. HRRN ist zu beachten, dass Tasks zu ihrem Ankunftszeitpunkt sofort gescheduled werden können. Gibt es zu einem Zeitpunkt mehrere Tasks mit höchster Priorität, so soll der Task mit der niedrigsten Nummer ausgeführt werden (also beispielsweise P4 vor P5).

Markieren Sie in den nachstehenden Vorlagen zu jedem Zeitpunkt den jeweils aktiven Task. Kennzeichnen Sie die *Arrival Time* jedes Tasks mit einem Pfeil "→" und den Zeitpunkt, zu dem ein Task seine gesamte *Service Time* konsumiert hat, mit "←". Eine Vorlage dient als Ersatz. Streichen Sie gegebenenfalls die falsch ausgefüllte Vorlage deutlich durch.

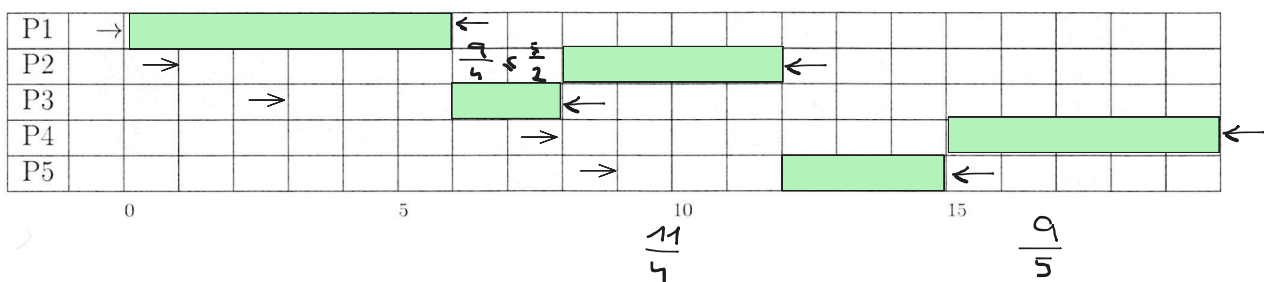
Scheduling nach dem **RR**-Verfahren:



Scheduling nach dem **SRT**-Verfahren:



Scheduling nach dem **HRRN**-Verfahren:



3) Fragen zu Betriebssystemen (40)

Was versteht man unter einem *Microkernel*? Welche zentralen Services muss ein Microkernel zur Verfügung stellen? (4)

Ein Kernel der minimal gehalten wird, wo alle Systemthreads als User level Threads gerannt werden.

Thread library.

Alles läuft als Userprozesse.

- Nur die nötigsten Basisservices befinden sich im Kernel, nicht zentrale Services des OS sind als Server Prozesse -> auf Prozessebene
- Vorteil: ist flexibel, einheitlich und Portabel
- Nachteil: Micro ist ungleich micro (nicht klar definiert, unterschiedliche Anzahl an Services)
- Zentrale Services die gestellt sein müssen:
 - Process switching
 - memory management
 - interrupts
 - hardware access
 - Nachrichtenaustausch

Was versteht man unter einem *Process Control Block*? Beschreiben Sie, aus welchen Teilen der PCB besteht und welche Informationen in diesen Teilen jeweils verwaltet werden. (5)

Ist Teil des Process Images und enthält Daten die das OS braucht um den Prozess zu verwalten wie:

- Process ID
- Processor State Information
- Process Control Information

Erklären Sie die Begriffe *Process Switch* und *Mode Switch*, sowie die Beziehung, in der diese beiden Konzepte stehen. Zählen Sie weiters die drei Klassen von Ereignissen auf, die einen Mode Switch nach sich ziehen. (4)

Nennen Sie die Bedingungen für das Eintreten eines Deadlocks und erklären Sie diese. (5)

Welche Schritte sind notwendig, um eine Datei, deren absoluter Pfadname gegeben ist, im Filesystem zu finden? (2)

Bei der Realisierung von Dateisystemen gibt es verschiedene Möglichkeiten, um die zu einer Datei gehörenden Datenblöcke zu organisieren bzw. auffindbar zu machen (Block-Allokierung). Nennen Sie vier verschiedene Strategien zur Block-Allokierung von Dateien und beschreiben Sie diese mit ihren Vor- und Nachteilen. (5)

Was versteht man unter dem Begriff *Relocation*? Wofür ist Relocation von Bedeutung? (2)

Beschreiben Sie Aufgabe und Funktion eines *Translation Lookaside Buffers*? Worauf hat man bei der Betriebssystemimplementierung bei einem Process Switch zu achten, wenn man einen Translation Lookaside Buffer verwendet? (3)

Welches Dateiformat einer regulären Datei hat für ein Betriebssystem besondere Bedeutung? Warum ist dieses Format wichtig? Wie wird es erkannt? (2)

Mit welcher logischen Struktur des I/O Systems versucht man bei der Realisierung von I/O Funktionen sowohl ein einheitliches Programmierinterface, als auch eine möglichst gerätespezifische Ansteuerung zu erreichen? (4)

Nennen Sie die drei Kategorien von *Security Threats* und beschreiben Sie diese. Geben Sie für jede Kategorie an, welches grundlegende Security-Ziel dadurch bedroht wird. (4)